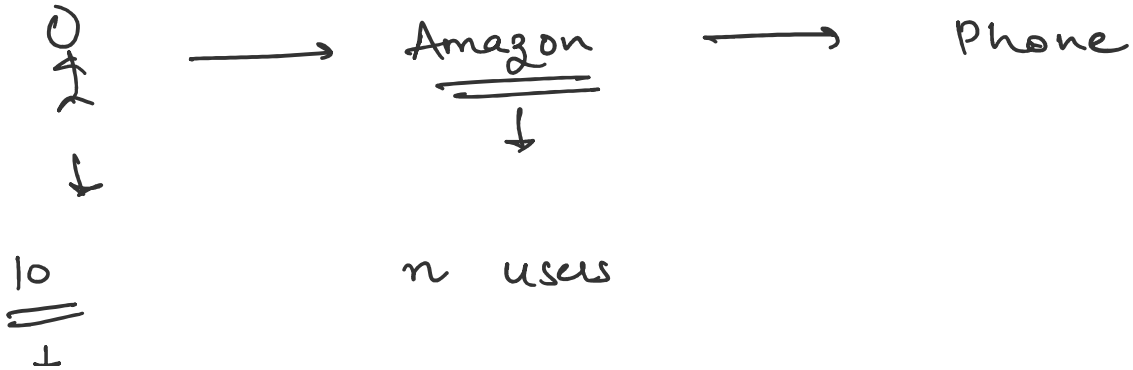
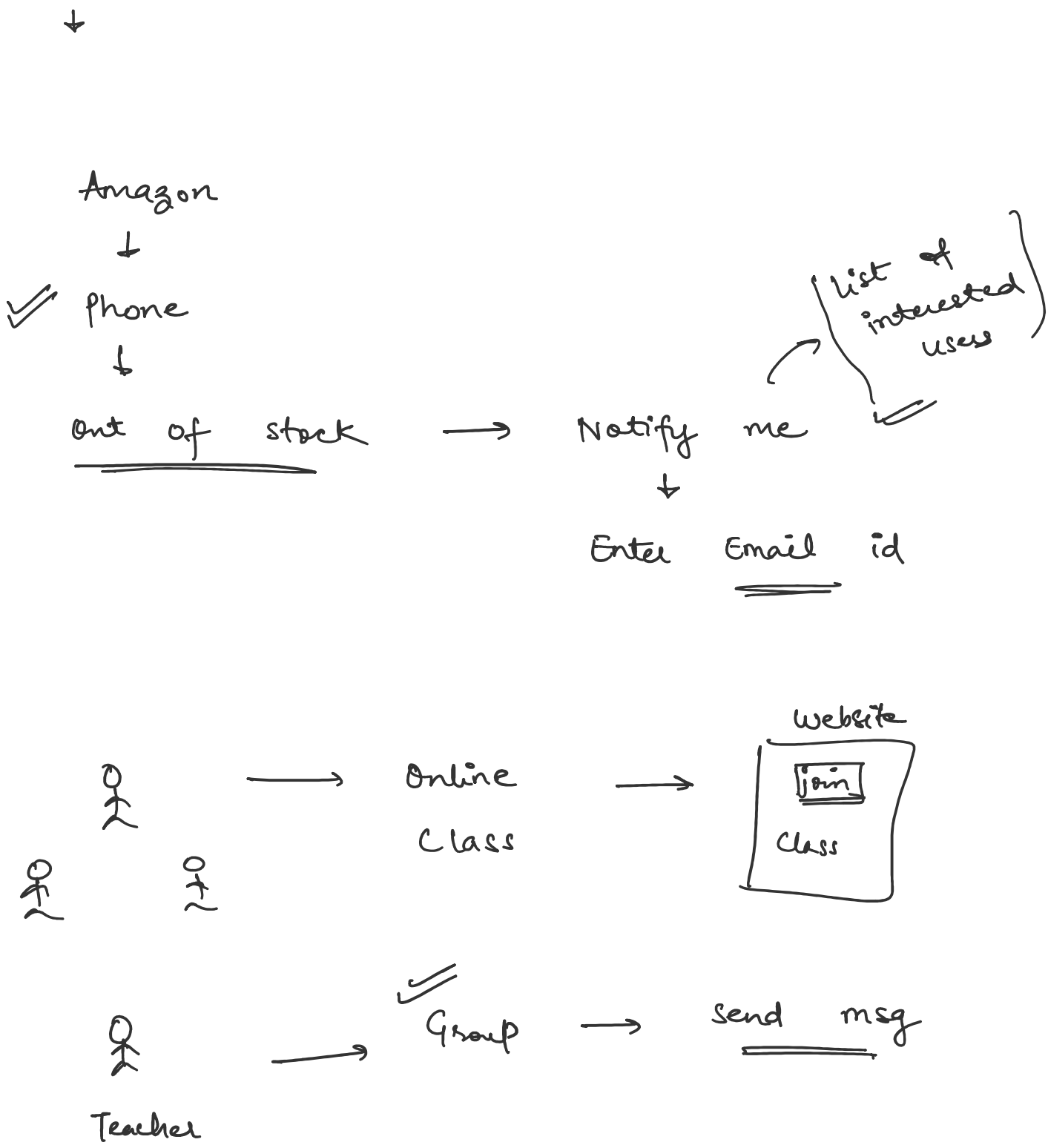
Class 16

Tuesday, July 14, 2026 9:37 PM

Observer Design Pattern

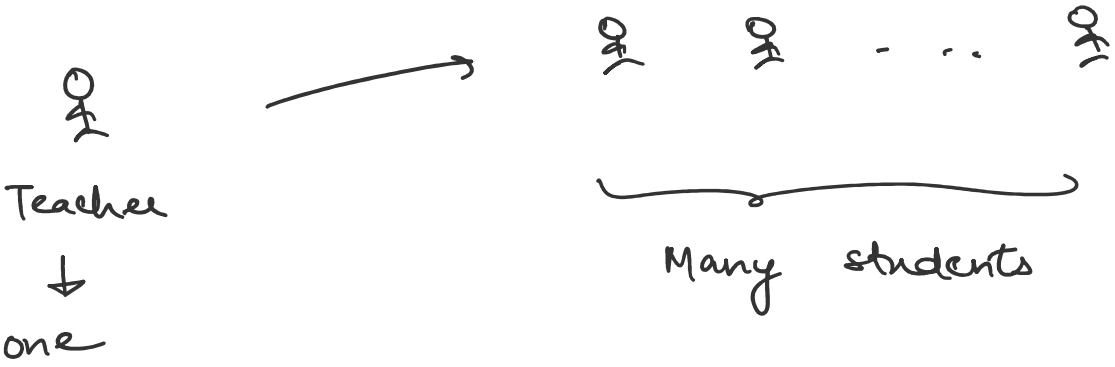




Soln → observer design pattern

one to many relationship

one to many relationship



Publisher

Subject → Object

→ keeping track of multiple

observers → Subscriber

students

Online class Notification system

Observer → Student

update (classTitle)

Subject → registerObserver ( )

→ removeObserver ( )

→ notifyObservers ( )

Concrete Subject → ClassScheduler

↓

implement Subject

↓

List of interested students

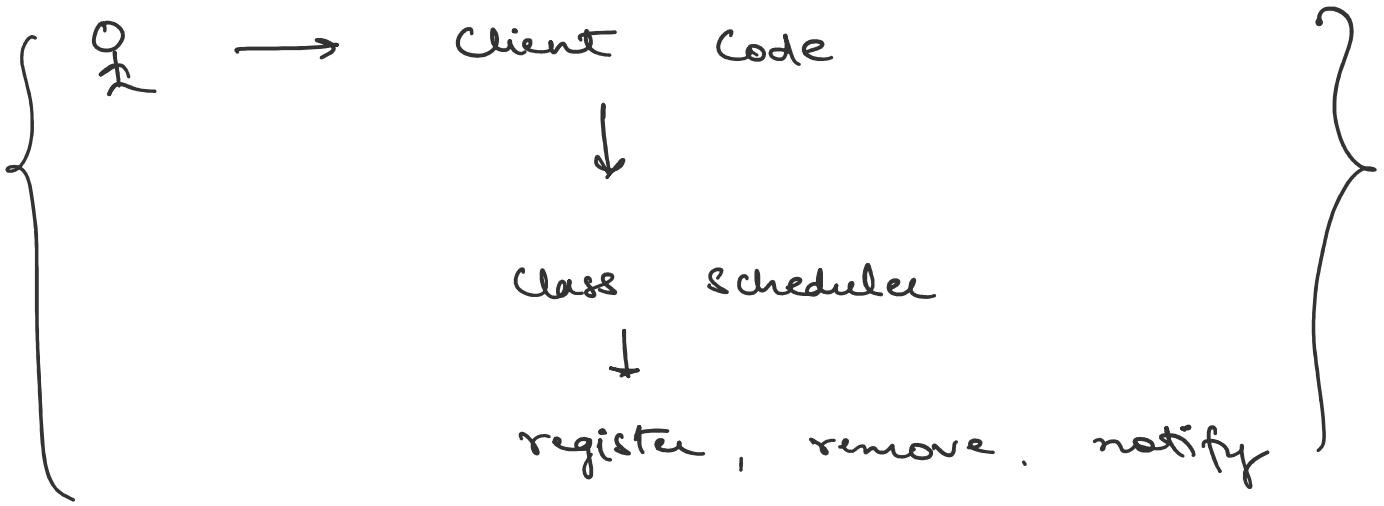
Concrete Observer → Student

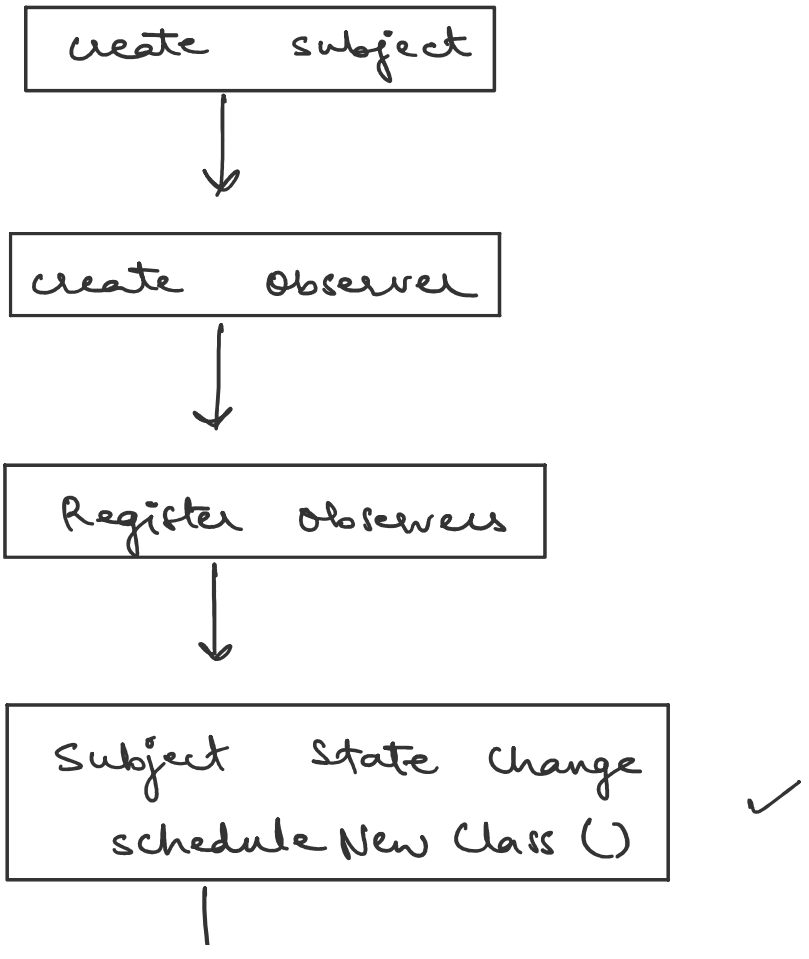
↓

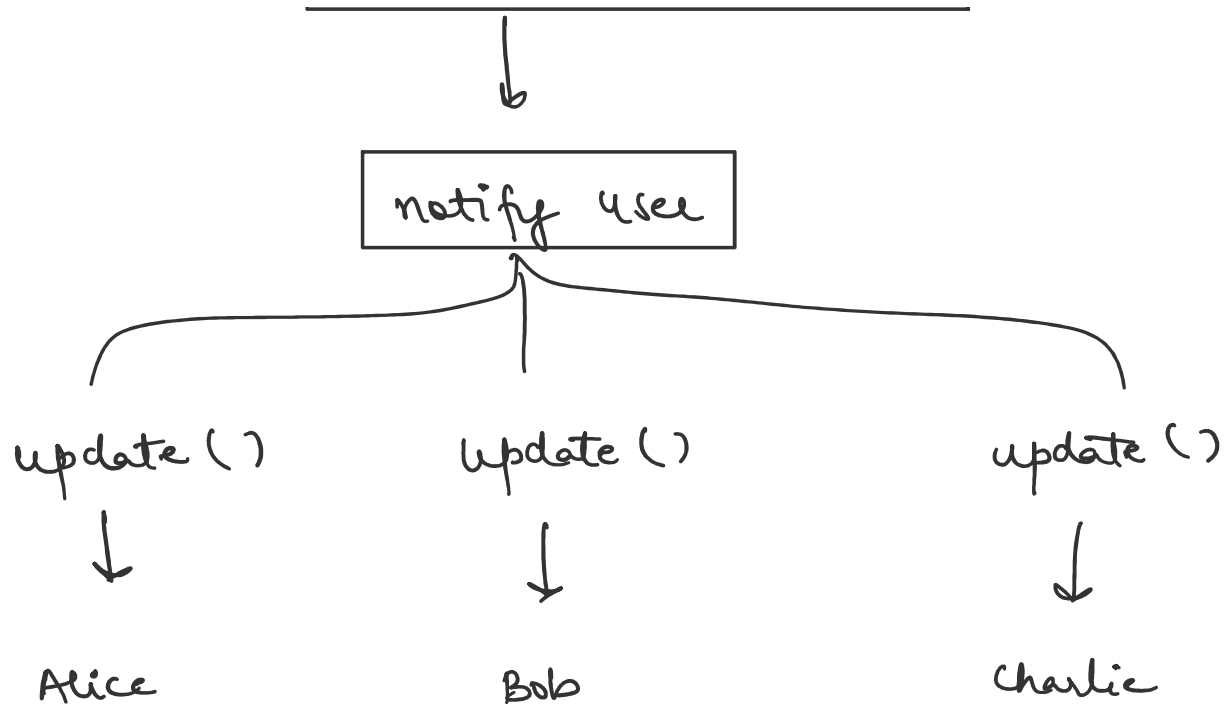
Define how a student responds to the notifications

↓

implement observer







food Delivery Appⁿ (Swiggy | Zomato)

Order ⟶ Preparing

↓

order prepared

↓

Picked up

↓

Out for delivery

 → Customer App^n

↓

To inform user

 → Delivery Partner App^n

↓

when order is ready

for pick up

 → Resturant Dashboard

↓

Order

Notification service → push notification

→ Email

→ SMS

Analytics service → Metrics

Notes :

Imagine you are building an **online learning platform**. Whenever a **new class is scheduled**, all subscribed students should receive a notification immediately.

How can we design the system so that every interested student is notified automatically whenever a new class is added?

Naive Solution

One approach is to make every student repeatedly check the website for new classes.

This has several problems:

- Students have to check the website again and again.
- The server receives many unnecessary requests.
- Students may miss important updates.
- The system becomes inefficient as the number of students grows.

We need a better solution.

The Solution: Observer Pattern

Instead of students checking for updates, the learning platform should notify all subscribed students automatically whenever a new class is scheduled.

This is exactly what the **Observer Pattern** does.

What is the Observer Pattern?

The **Observer Pattern** is a **Behavioral Design Pattern** that creates a **one-to-many relationship** between objects.

One object, called the **Subject**, keeps track of multiple **Observers**. Whenever the Subject's state changes, it automatically notifies all registered Observers.

In simple words: **"One object changes, and all interested objects are notified automatically."**

Components

- **Subject (Publisher)** – Maintains a list of observers, allows observers to register and unregister and sends notifications whenever its state changes.

- **Observer (Subscriber)** – Registers with the subject, receives updates whenever the subject changes and performs its own action after receiving the update.

How It Works

1. Observers register with the Subject.
2. The Subject maintains the list of registered observers.
3. The Subject's state changes.
4. The Subject notifies all registered observers.
5. Each observer performs its own task independently.

Real-Life Use Cases

The **Observer Pattern** is used whenever one object changes and multiple other objects need to react automatically. Some common examples are:

1. YouTube Notifications
When a creator uploads a new video, all subscribers receive a

notification.

- **Subject:** YouTube Channel
- **Observers:** Subscribers

2. Online Learning Platform

When a new class is scheduled, all enrolled students receive a notification.

- **Subject:** Class Scheduler
- **Observers:** Students

3. Weather Monitoring System

When the temperature changes, multiple applications update automatically.

- **Subject:** Weather Station
- **Observers:** Mobile App, Smartwatch, Website

4. Stock Market Application

When a stock price changes, all interested applications receive the latest price.

- **Subject:** Stock Exchange
- **Observers:** Trading App, Dashboard, Alert Service

5. GUI Event Listeners

When a user clicks a button, all registered event listeners are notified.

- **Subject:** Button
- **Observers:** ActionListener, MouseListener

6. Social Media Notifications

When a user creates a new post, followers receive updates in their feed or notification panel.

- **Subject:** User Account
- **Observers:** Followers

7. Chat Applications
When a new message is sent in a group, all group members receive the message instantly.

- **Subject:** Chat Room
- **Observers:** Group Members

Implementation of Online Class Notification System

Build an online learning platform where:

- Students receive notifications whenever a new class is scheduled.
- Students can subscribe or unsubscribe from notifications.
- The scheduler should not depend on concrete student classes.
- New observer types should be easy to add.

The **Observer Pattern** is a perfect fit for this requirement.

Step 1: Create the Observer Interface
The **Observer** interface represents any object that wants to receive notifications.

```
public interface Observer {
    void update(String classTitle);
}
```

Whenever a new class is scheduled, the Subject calls update() on every registered observer.

Step 2: Create the Subject Interface
The **Subject** manages all observers.

```
public interface Subject {
    void registerObserver(Observer observer);
    void removeObserver(Observer observer);
    void notifyObservers();
}
```

Responsibilities:

- Register observers
- Remove observers
- Notify all observers when the state changes

Step 3: Create the Concrete Subject (ClassScheduler)
The **ClassScheduler** maintains the list of subscribed students and notifies them whenever a new class is scheduled.

```
import java.util.*;

public class ClassScheduler implements Subject {
    private final List<Observer> observers = new ArrayList<>();
    private String classTitle;
```

```
    @Override
    public void registerObserver(Observer observer) {
        observers.add(observer);
    }

    @Override
    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }

    @Override
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(classTitle);
        }
    }

    public void scheduleNewClass(String classTitle) {
        this.classTitle = classTitle;
        notifyObservers();
    }
}
```

The scheduler knows **who** to notify, but it does not know **how** each observer handles the notification.

Step 4: Create the Concrete Observer (Student)

The **Student** class implements the **Observer** interface and defines how a student responds to notifications.

```
public class Student implements Observer {
    private final String name;
```

```
    public Student(String name) {
        this.name = name;
    }

    @Override
    public void update(String classTitle) {
        System.out.println("Hi " + name + ", a new class has been
scheduled: " + classTitle);
    }

    @Override
    public String toString() {
        return name;
    }
}
```

Each student decides how to handle the notification.

Step 5: Client Code

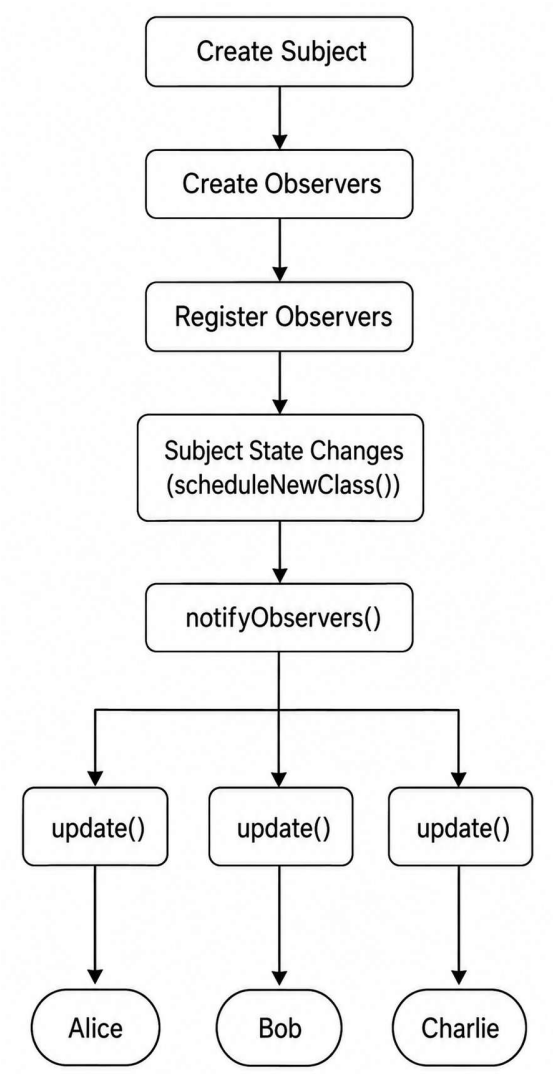
The client creates the Subject, registers observers, and schedules new classes.

```
public class Main {
    public static void main(String[] args) {
        ClassScheduler scheduler = new ClassScheduler();
        Student s1 = new Student("Alice");
        Student s2 = new Student("Bob");
        Student s3 = new Student("Charlie");
        Student s4 = new Student("Jessica");

        scheduler.registerObserver(s1);
        scheduler.registerObserver(s2);
```

```
        scheduler.registerObserver(s3);
        scheduler.scheduleNewClass("System Design - Observer
Pattern");

        System.out.println("------------------------------");

        scheduler.registerObserver(s4);
        scheduler.removeObserver(s2);
        scheduler.scheduleNewClass("Advanced Java
Programming");
    }
}


/* *

    Output :
        Hi Alice, a new class has been scheduled: System Design -
Observer Pattern
        Hi Bob, a new class has been scheduled: System Design -
Observer Pattern
        Hi Charlie, a new class has been scheduled: System
Design - Observer Pattern
        ------------------------------
        Hi Alice, a new class has been scheduled: Advanced Java
Programming
        Hi Charlie, a new class has been scheduled: Advanced
Java Programming
        Hi Jessica, a new class has been scheduled: Advanced
Java Programming

* */
```

How the Observer Pattern Works Internally

The complete flow is shown below:



When the Subject's state changes, it loops through all registered observers and calls their update() method. Since the Subject depends only on the **Observer** interface, new observer types can be added without modifying the Subject.